université de BORDEAUX

Automatic tuning of the IO500 benchmark parameters with Bayesian Optimization

M2 Computer Science
Internship Report

Inria Centre at the University of Bordeaux

Supervisors:

August 26, 2026

Contents

1. Introduction **1**
1.1. Objectives of the internship 3
1.2. Contributions 3
1.3. Organisation of the report 3

2. Context of the internship **3**
2.1. Inria and the Inria Centre at the University of Bordeaux 3
2.2. The host team : TADaaM [6] 4
2.3. Scientific and technical environment 4
2.4. Objectives and specifications 5
2.5. Organisation of the work 6

Bibliography **7**

1. Introduction

High Performance Computing (HPC) systems are used to run large-scale scientific and engineering applications. The applications use many compute resources in parallel and often process large amounts of data. Therefore, an application's performance does not only depend on the computing power of the system, but also on the performance of its storage infrastructure. In many HPC workloads, I/O operations become the bottleneck that limits overall application performance.

Parallel file systems (PFS) such as Lustre [10], BeeGFS [15] and GPFS [9] are commonly used to provide high-performance storage in HPC environments. In a PFS, the storage capacity is exposed as a set of logical units called Object Storage Targets (OSTs). When a file is written, it is split into fixed-size chunks that are sent in parallel to several targets, so that the bandwidth of many storage devices is used at once. This process is known as striping and is illustrated in Figure 1. How many targets a file is spread over, called the stripe count, and how large each chunk is, called the stripe size, are configuration choices rather than fixed properties of the system. Therefore, a PFS exposes configuration parameters that can affect I/O performance. Beyond striping, performance can also depend on the transfer size, the number of processes, or the placement of processes on compute nodes. The best values for these parameters are not necessarily the same for every machine: a configuration that performs well on one system may perform poorly on another one.

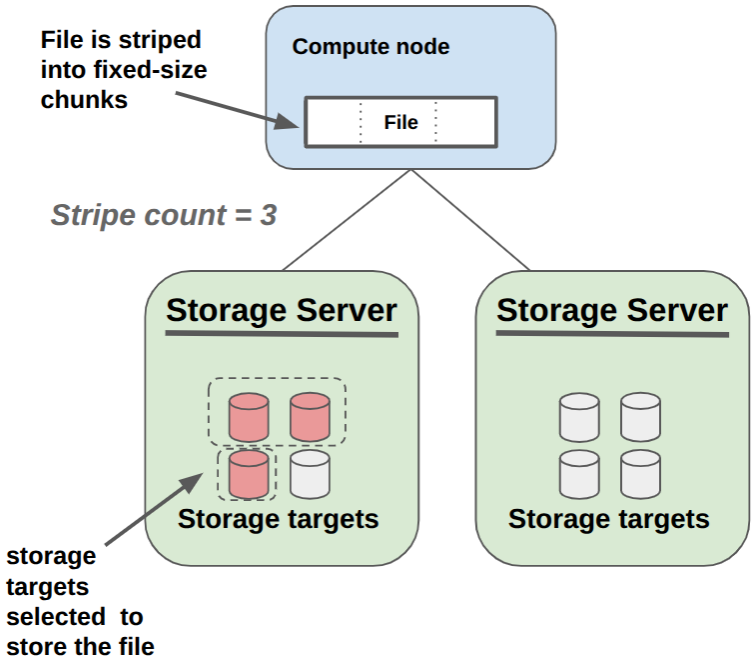


Figure 1: Striping in a parallel file system. A file created on a compute node is split into fixed-size chunks, which are written in parallel to storage targets hosted by the storage servers. Here the stripe count is three, so the file is spread over three of the eight available targets.

To evaluate the performance of these systems, we rely on benchmarks. A benchmark is a program designed to stress a specific part of a system under controlled and reproducible conditions, so that

the measurements it produces can be compared across machines and configurations [2]. For I/O, two of the most widely used benchmarks are IOR [12], which measures the bandwidth reached by large parallel read and write operations, and mdtest [13], which measures the rate at which metadata operations such as creating, opening and removing files are performed. A different approach is taken by mini-applications, or miniapps [1], which reproduce the access pattern of a real scientific application in a reduced and self-contained form, and therefore measure performance under a workload closer to production use. Each of these benchmarks characterises one aspect of the system, and none of them alone describes the storage system as a whole.

Thus, the variety of parameters and access patterns makes the evaluation of I/O performance difficult: a benchmark must not only measure the performance of the storage system, but also provide a meaningful way to compare different systems and configurations. For this reason, the HPC community maintains rankings in which every system runs the same benchmarks under a common set of rules, so that the resulting scores can be compared and ordered. The best known of them is the TOP500 [19], which ranks HPC platforms by their computational performance. In the I/O world, the community created the IO500 benchmark suite [7]. IO500 is widely used to evaluate and rank the I/O performance of HPC storage systems, and it combines several workloads, including bandwidth-oriented and metadata-oriented operations, into a single score, which is used to produce the IO500 ranking.

Because IO500 is a ranking, there is a direct incentive for HPC sites to obtain the best possible score. For vendors and system integrators, a high position is a commercial argument, used in marketing material and in procurement discussions. For computing centres, it is a public and comparable demonstration that the machine they operate delivers the capability it was built for, which matters both for the visibility of the centre and for justifying the investment it represents. However, the reported performance of a system does not only depend on its hardware, but also on how the system is configured and how the benchmark is executed. Finding a good configuration is therefore essential to obtain the highest possible score.

However, there is a large number of possible configurations, which makes the tuning of IO500 difficult: a configuration can involve several parameters, and each parameter can have multiple possible values. The combination of these parameters creates a potentially large search space, and finding a good configuration therefore requires evaluating many possible combinations.

Another important difficulty is the cost of an IO500 evaluation. According to the IO500 submission rules [8], each write and create phase must run for at least 300 s. A run that finishes the write phase earlier is considered invalid. This requirement makes a valid IO500 measurement relatively expensive in terms of cluster time: on the target platform, one valid measurement takes approximately 30 min of exclusive cluster time. When several configurations and repetitions are considered, the total cost becomes very large. For instance, in the parameter space considered in this work, approximately 7,840 configurations have to be evaluated. Measuring all of them with three repetitions would require approximately 12 000 h of wall-clock time, which corresponds to more than one year of computation on the target machine. Therefore, An exhaustive search is not practical.

This leads to the main research question of this internship:

> *Can a smarter search method find a near-optimal IO500 configuration using far fewer evaluations, and which search method is most effective for this problem?*

1.1. Objectives of the internship

The objective of this internship is to tune IO500 with a guided search instead of an exhaustive one, and to determine which search method should be used in practice. Bayesian Optimization is the main candidate, but it is a family of methods rather than a single one, so a large part of the work consists in comparing its variants, and comparing them against random and exhaustive search.

The work is carried out with IOPS [11], a benchmark orchestration framework developed in the team. IOPS describes a parametric study in a single YAML file and takes care of generating the runs, submitting them to the cluster scheduler, parsing their output and caching the results. Two of its features matter here: the search strategy is a configuration option, so exhaustive, random and Bayesian searches can be compared without changing anything else; and a study can be replayed from the cache, which is what makes the offline comparison possible.

1.2. Contributions

The main contributions of this work are the following.

- *A cheap ground-truth methodology* to evaluate search strategies for tuning IO500, based on a set of invalid scores that are much cheaper to generate but still allow the strategies to be compared.
- *A systematic comparison of Bayesian Optimization variants*, which identifies the setup that performs best and shows which of its parameters actually matter.
- *A validation campaign* on the cluster, checking that the configurations selected on the cheap proxy remain the best ones under the official IO500 rules.

1.3. Organisation of the report

The remainder of this report is organised as follows. The next section presents the host laboratory and the environment in which the internship took place. The background section then introduces what is needed to read the rest of the report: the IO500 benchmark and its rules, the parameters exposed by parallel file systems, and the principles of Bayesian Optimization. Related work on I/O auto-tuning is reviewed next, followed by a description of the tools, the platform and the experimental setup. The methodology section explains how the ground truth was built and how the search strategies were compared, and the results section reports and analyses the measurements. The report closes with a discussion of the limitations of the study and a conclusion.

2. Context of the internship

2.1. Inria and the Inria Centre at the University of Bordeaux

Inria is the French national research institute for digital science and technology. Its research activities cover a wide range of topics in computer science, applied mathematics, and digital technologies. Research is organised into project teams, which bring together researchers, engineers, PhD students, and other scientific staff around common research topics.

The internship was carried out at the Inria Centre at the University of Bordeaux. The centre is located on the Talence campus and brings together research teams working on different areas of

digital science. Inria currently has 22 project teams associated with the centre, including 20 teams located in Talence and two teams in Pau. The centre brings together more than 470 researchers, engineers, technical staff, and other members of the research and innovation community. These teams are developed in partnership with institutions such as the University of Bordeaux, Bordeaux INP, CNRS, Inserm, INRAE, and other academic and industrial partners [3].

The research activities of the centre cover several areas of digital science. These include high-performance computing and Big Data, machine learning, and digital health. This diversity reflects the different research problems addressed within the centre, ranging from fundamental computer science to applications in science, engineering, and health. For the work presented in this report, the high-performance computing research activities are particularly relevant.

Several project teams at the centre work on high-performance and distributed computing. For example, the STORM, TADaaM, CONCACE, and TOPAL teams are associated with research topics related to high-performance computing, systems, optimisation, and the execution of parallel applications [5]. These teams contribute to a local research environment in which computing performance, system software, and large-scale data management are studied using both theoretical approaches and experiments on real HPC platforms.

2.2. The host team : TADaaM [6]

The internship was carried out within the TADaaM project team at the Inria Centre at the University of Bordeaux. TADaaM works on data management for high-performance computing systems, with a particular interest in the interaction between applications, computing resources, networks, and storage systems. The team studies how data can be managed efficiently at system scale and how the characteristics of the underlying infrastructure affect application performance.

The internship is part of the team's activities related to storage performance characterisation and I/O benchmarking. In particular, the work contributes to the study of how HPC storage systems can be configured and evaluated using the IO500 benchmark suite. The internship also makes use of IOPS, a benchmark orchestration and optimisation framework developed within the team. The framework provides the tools needed to automate benchmark executions, explore different parameter configurations, and collect and analyse their results.

The internship was supervised by Luan Teylo, a researcher working within the TADaaM team. The work was also connected to a broader collaboration around IO500 and HPC I/O performance. Discussions involved Liam Radita, Sara Neuwirth, and Jay Lofstead. These interactions provided feedback on the benchmark configuration, experimental methodology, and interpretation of the results.

The specific role of the internship was therefore to investigate the automated tuning of IO500 configurations. The work focused on the experimental side of this research activity, from running the benchmark on the target platform to evaluating automated search strategies and validating their results.

2.3. Scientific and technical environment

The experiments conducted during the internship were carried out on PlaFRIM, the experimental computing platform used by the Inria research community in Bordeaux. PlaFRIM provides access

to several types of computing nodes and is designed to support experiments on high-performance computing systems and new hardware technologies [4].

The experiments presented in this report were conducted on the `bora` nodes, which consist of 44 nodes, from `bora001` to `bora044`. Each node is equipped with two 18-core Intel Xeon Gold 6240 processors, providing 36 physical CPU cores per node, and 192 GB of memory. The nodes are connected through a 100 Gbit/s Omni-Path network, as well as a 10 Gbit/s Ethernet network. For the experiments described in this report, the BeeGFS parallel file system is accessed through the 100 Gbit/s Omni-Path network [4].

The use of a dedicated and homogeneous set of nodes is important for the experiments carried out in this work. Since the objective is to compare different IO500 configurations, changes in the measured performance should primarily result from the parameters being studied rather than from differences between compute nodes. [4].

The experiments were managed using SLURM, the workload manager used on PlaFRIM. SLURM was used to request computing resources and to execute benchmark experiments on the selected nodes. Depending on the experiment, jobs could therefore be allocated with a specified number of nodes and resources. This was particularly important for the internship because the number of compute nodes and processes is itself part of the parameter space explored when tuning IO500.

The storage system used for the experiments is BeeGFS, a parallel file system designed to provide high-performance access to shared storage. The benchmark data and results were stored on the BeeGFS file system provided by PlaFRIM. In addition to BeeGFS, the `bora` nodes provide a local 1 TB disk mounted under `/tmp`. The experiments described in this report, however, focus on the performance of the parallel storage system rather than the local disks [4].

The software environment consisted of Linux together with the tools required to build and execute the benchmarks. Python 3.10 was used for the development and execution of the scripts associated with the experiments. OpenMPI was used as the MPI implementation required by the parallel benchmark workloads. The main research tools were IO500, used to evaluate the I/O performance of the system, and IOPS, used to automate the execution and exploration of different benchmark configurations.

The development process was split between the PlaFRIM environment and a local development machine. The source code, scripts, and experimental configurations were managed using Git and GitLab. This allowed the development work and experimental results to be kept under version control while the computationally intensive benchmark executions were performed on the cluster.

2.4. Objectives and specifications

The main objective of the internship was to tune the IO500 benchmark on the PlaFRIM `bora` cluster and determine whether automated search strategies could find good configurations with fewer evaluations than an exhaustive search.

From the beginning, the main elements of the study were fixed: IO500 was used as the benchmark, PlaFRIM `bora` as the target platform, and IOPS as the framework for running the experiments. Bayesian Optimization (BO) was selected as the main search strategy, with random search and exhaustive search used for comparison.

The first stage of the internship focused on the exhaustive search. The objective was to explore the parameter space and obtain measurements that could later be used to compare the different search strategies. Each configuration was also evaluated several times to account for the variability of HPC measurements.

The study then focused on comparing the different search strategies under a limited number of evaluations. In particular, the objective was to determine whether BO could find configurations close to the best configurations found by exhaustive search while requiring significantly fewer evaluations.

As the internship progressed, the scope was extended to the development of IOPS. A plugin for running IO500 experiments was added to IOPS, together with an example YAML configuration describing the benchmark and the parameters explored during the study. This made it possible to run the IO500 experiments directly through IOPS and provided a reproducible configuration for the experiments.

The IOPS documentation was also updated to describe the new functionality and its usage.

The search component of IOPS was also extended with an implementation of the Tree-structured Parzen Estimator (TPE). At this stage, the method had been implemented but had not yet been evaluated on the IO500 experiments. Other optimisation methods, such as Iterated Racing (irace) and Covariance Matrix Adaptation Evolution Strategy (CMA-ES), were also considered as possible extensions.

These additional tasks emerged as the project evolved and extended the initial scope of the internship.

2.5. Organisation of the work

The internship took place from 14 April to 16 September. The work was organised progressively, starting with the study of the existing tools and the experimental environment, followed by the experimental campaigns and the development work.

The work was supervised by Luan Teylo through a weekly meeting of approximately 30 minutes. These meetings were used to review the progress of the work, discuss experimental results, and define the next tasks. Notes and follow-up tasks were recorded using the Inria Notes platform. Mattermost was also used for communication.

The work was managed using Git and GitLab. A personal repository was used to keep track of the work carried out during the internship, while the IOPS team repository was used for contributions to the existing IOPS project. This allowed the development work and experimental scripts to be versioned throughout the internship.

The internship also included a meeting with Liam Radita, Sara Neuwirth, and Jay Lofstead as part of the broader collaboration around the IO500 work.

Bibliography

References

[1] Michael A. Heroux, Douglas W. Doerfler, Paul S. Crozier, James M. Willenbring, H. Carter Edwards, Alan Williams, Mahesh Rajan, Eric R. Keiter, Heidi K. Thornquist, and Robert W. Numrich. Improving performance via mini-applications. Technical Report SAND2009-5574, Sandia National Laboratories, 2009. VERIFY: author list and report number.

[2] Torsten Hoefler and Roberto Belli. Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2015. VERIFY: article number / pages.

[3] Inria. Inria centre at the university of bordeaux. `https://www.inria.fr/en/inria-centre-university-bordeaux`. Accessed: 2026-08-24.

[4] Inria. Plafrim – cluster of inria of bordeaux. `https://plafrim-users.gitlabpages.inria.fr/doc/`. Accessed: 2026-08-24.

[5] Inria. The project-teams – distributed and high performance computing. `https://www.inria.fr/en/list-of-project-teams`. Accessed: 2026-08-24.

[6] Inria. Tadaam – topology-aware system-scale data management for high-performance computing. `https://www.inria.fr/en/tadaam`. Accessed: 2026-08-24.

[7] IO500 Foundation. The IO500 list. `https://io500.org/`, 2026. Accessed: August 26, 2026.

[8] IO500 Foundation. IO500 submission rules. `https://io500.org/rules/submission`, 2026. Accessed: August 26, 2026.

[9] Frank Schmuck and Roger Haskin. GPFS: A shared-disk file system for large computing clusters. In *Proceedings of the 1st USENIX Conference on File and Storage Technologies (FAST)*, pages 231–244, 2002. VERIFY: page numbers.

[10] Philip Schwan. Lustre: Building a file system for 1000-node clusters. In *Proceedings of the 2003 Linux Symposium*, 2003. VERIFY: page numbers.

[11] TADaaM, Inria. IOPS: Integrated orchestration for parametric studies. `https://iops-benchmark.com`, 2026. Source repository: `https://gitlab.inria.fr/hpc_io/iops`. Accessed: August 26, 2026.

[12] The IOR contributors. IOR and mdtest: Parallel filesystem I/O benchmarks. `https://github.com/hpc/ior`, 2026. Accessed: August 26, 2026.

[13] The IOR contributors. mdtest: Metadata benchmark for parallel file systems. `https://github.com/hpc/ior`, 2026. Distributed with the IOR suite. Accessed: August 26, 2026.

[14] The scikit-optimize contributors. scikit-optimize: Sequential model-based optimization in Python. `https://scikit-optimize.github.io/`, 2026. Version TO-COMPLETE.

[15] ThinkParQ GmbH. BeeGFS: The parallel cluster file system. https://www.beegfs.io/, 2026. Accessed: August 26, 2026.

[16] TO-COMPLETE. To-complete: 2022 I/O performance study. https://inria.hal.science/hal-03753813/, 2022.

[17] TO-COMPLETE. To-complete: IO500 tuning on CLAIX23. In *IEEE International Conference on Cluster Computing Workshops (CLUSTER Workshops)*, 2025.

[18] TO-COMPLETE. To-complete: IO500 benchmark reference paper, 2026.

[19] TOP500.org. The TOP500 list. https://www.top500.org/, 2026. Accessed: August 26, 2026.